🔌 Plugins y MCPs los superpoderes del agente

Codificar con agentes · Parte 5 · Octubre 2026

👤 Sergi Garcia Barea

CC BY-SA 4.0

🚫 Cuando el agente solo puede leer y escribir

😐 Agente básico

- No ve el issue con los requisitos
- No puede abrir el PR con su cambio
- No sabe si la BD ya tiene esa tabla
- No puede probar la web que acaba de hacer

🚀 Agente conectado

- Lee los issues y los docs oficiales
- Abre PRs comentado cada decisión
- Consulta el esquema real de la base de datos
- Abre su propia web y verifica el DOM

💡 La diferencia se llama **herramientas**: las capacidades extra que se le conectan al agente.

🔊

🔗 MCP: el "USB" de los modelos

MCP (Model Context Protocol) es un estándar abierto: cada servidor MCP expone herramientas que *cualquier* agente compatible puede usar.

🧰 Un servidor = un dominio

GitHub, PostgreSQL, navegador, sistema de ficheros, documentación de librerías… Hay cientos publicados.

🤖 El agente decide cuándo

Tú le pides la tarea; si necesita consultar la BD o el issue, usa la herramienta. Tú apruebas cada uso.

🔀 Funciona igual en todos

El mismo servidor MCP sirve en OpenCode, Claude Code, Codex o Antigravity. Conectas una vez, usas siempre.

🚫 No es magia

Sigue siendo el agente de siempre, con más ojos y más manos. Por eso las reglas de revisión siguen valiendo.

🎯 Píldora clave del deck: **MCP = herramientas.** plugins = paquetes que traen skills + comandos + MCPs ya montados.

🧰 Los servidores MCP que más se usan en dev



GitHub

Issues, PRs, revisiones, releases. El repositorio como herramienta.



Base de datos

Esquema real, consultas de solo lectura, migraciones.



Navegador

Abrir la web, consola, DOM, capturas. Probar lo generado.



Docs

API y documentación de frameworks actualizada, no de memoria.

También: filesystem seguro, Git, Docker, Jira/Linear, Slack, Playwright, Grafana…

💡 Regla práctica: conecta los MCP del **dominio de tu tarea**. Tres servidores bien elegidos valen más que quince sin sentido.

⚙️ Cómo se activa un MCP

📝 CONFIG (DENTRO DEL PROYECTO)

```
// opencode.json (o equivalente)
{
  "mcp": {
    "github": {
      "type": "remote",
      "url":
"https://api.githubcopilot.com/mcp/",
      "enabled": true
    },
    "bd-local": {
      "type": "local",
      "command": [
        "uvx", "mcp-server-postgres",
        "postgresql://localhost/demo"
      ],
      "readonly": true,
      "enabled": true
    }
  }
}
```

OpenCode · MCP · Config declarativa

💡 DETALLES QUE IMPORTAN

- ✅ **local** (comando que se ejecuta en tu máquina) vs **remote** (URL de servicio)
- ✅ **readonly**: consulta, no modifica. Para BD, casi siempre sí
- ✅ Va al repo → el equipo comparte la misma configuración
- ✅ Secrets por variable de entorno, nunca en el JSON
- ✅ El agente lista sus herramientas: "¿qué MCPs tienes activos?"

🔊

🐙 Caso 1: GitHub MCP — del issue al PR

💬 PROMPT

```
"Lee el issue #42 'Total del
carrito'.

1. Plan corto: dónde tocar y qué tests
2. Implementa el cambio con tests
3. Commita en una rama nueva
4. Abre el PR citando el issue

Solo consulta GitHub para datos,
no publiques nada más."
```

OpenCode · MCP GitHub · Issue → PR

💡 LO QUE GANA EL FLUJO

- ✅ Los requisitos reales: no los que tú recordabas
- ✅ El PR describe el cambio y cita el issue automáticamente
- ✅ Otros devs comentan el PR: revisión humana intacta
- ✅ El agente nunca hace merge: eso sigue siendo tuyo
- ✅ Ideal para tareas pequeñas e independientes

🔊

🗄️ Caso 2: BD MCP — migrar sin adivinar

💬 PROMPT

```
"Queremos añadir 'favoritos' a
la app de recetas. Antes de proponer
el modelo, añade los esquemas
reales de la BD con tus herramientas
de consulta.

Dame: tablas relacionadas y claves,
tipos de dato que se usan aquí,
y el patrón de nombres existente.
Luego propón la migración."
```

OpenCode · MCP BD · Solo lectura

💡 POR QUÉ CAMBIA LA CALIDAD

- ✅ El modelo se adapta al esquema real, no a su imaginación
- ✅ Sin readonly: solo lee tablas y tipos
- ✅ Detecta conflictos de nombres antes de crear nada
- ✅ La migración resultante encaja a la primera
- ✅ Medidas enr contra: confirmar supuestos antes de escribir código

🔊

🌐 Caso 3: Navegador MCP — verificar lo escrito

💬 PROMPT

```
"El formulario nuevo está en
http://localhost:5173/registro.

Compruébame con el navegador:
1. Un submit con campos vacíos
   → ¿aparecen los errores?
2. Un email correcto → ¿se envía?
3. ¿Hay errores de consola?

Hazme una lista de lo que pasó
y de lo que falla."
```

OpenCode · MCP navegador · QA

💡 EL BUCLE SE CIERRA

- ✅ El agente escribe el código y lo prueba solo
- ✅ Ve las dos realidades: código y rendered
- ✅ Los errores de consola cuentan como fallos
- ✅ Menos "lo he probado mentalmente"
- ✅ Complementa a los tests: cubren UI real

🔊

📦 Plugins: capacidades en paquete

Un **plugin** agrupa lo que de otro modo montarías a mano: skills + comandos + configuración de MCPs. Se instala con un comando y queda lista para todo el equipo.

/filesystem? Ejemplo: plugin de code-review

Skill de reglas de revisión + comando `/review` + MCP de GitHub. Instalado una vez, disponible en cada sesión.

🧪 Ejemplo: plugin de testing

Skills "pytest" y "coverage" con las recetas del stack + comando para lanzar la suite con el MCP de Docker.

🛠️ Cliente: listar e instalar

```
opencode plugins list
opencode plugins install review-pack
```

Los plugins concretos dependen del agente; el concepto es común a varios.

🧭 Cuando conviene

Cuando un flujo se repite y ya lo has explicado dos veces: conviértelo en skill, comando y MCP empaquetados.

💡 Relacionado: **skills** = saber hacer una tarea · **MCPs** = herramientas nuevas · **plugins** = paquetes que traen ambas.

🛡️ Riesgos de conectar herramientas

🔑 Permisos que sobran

Un MCP de GitHub con permisos de borrado "por si acaso" = un incidente esperando. Solo lo mínimo.

🚭 Servidores no verificados

Mira quién mantiene cada servidor y qué datos pueden salir. Instala de fuentes conocidas.

🗄️ BD de escritura por defecto

Para explorar, readonly. La BD de producción no se conecta: se conecta una copia.

👁️ Aprobación de cada uso

Por defecto, el agente pregunta antes de crear un issue o ejecutar una consulta. No desactives eso por comodidad.

🧠 Demasiadas herramientas

Con 30 MCPs activos, el agente se despista y elige la herramienta equivocada. Menos es más.

🧪 Entorno aislado

El comando de un MCP local corre en tu máquina: usa contenedores para lo que no dominas.

🔊

🧭 Mini-caso completo: pantalla de favoritos

Todo lo de las partes 2–5 combinado en una tarea real de 20 minutos.

🪜 **1. Plan** · "Solo plan para favoritos: modelo, endpoint, pantalla y criterios."

🗄️ **2. MCP BD** · El agente consulta el esquema y ajusta la migración al patrón real.

💻 **3. Hacer** · Paso a paso: migración → endpoint con tests → pantalla. Commits verificados.

🌐 **4. MCP navegador** · El agente abre localhost, prueba el flujo y reporta lo que falla.

🐙 **5. MCP GitHub** · Rama + PR citando el issue, listo para review humana.

🧠 **6. Aprender** · Regla nueva a AGENTS.md: "los favoritos se guardad con el patrón X".

🎯 El agente no solo escribe más rápido: aquí el flujo completo es el multiplicador.

🔊

✨ En esta parte 5

🔗 MCPs

Herramientas estándar: GitHub · BD (readonly) · navegador · docs — mismo estándar para todos los agentes

📦 Plugins

Paquetes con skills + comandos + MCPs, para flujos repetidos de equipo

🛡️ **Principio de mínimo privilegio**: permisos justos, BD demo read-only, aprobación de cada uso

🎛️ **Menos es más**: conecta solo el dominio de tu tarea; suspende el resto

Última: **Dev con agentes** — configuraciones, pipelines y operación, todo generado con el copiloto. 🚀

CC BY-SA 4.0 · Sergi Garcia Barea · Octubre 2026

🔊